창의적 문제해결(CPS) 스캐폴딩 AI 에이전트

1. 개요

본 문서는 대학생 예비교사를 대상으로 창의적 메타인지를 촉진하는 AI 에이전트 시스템의 Backend 구현 방법론과 작동 원리를 기술합니다. 본 시스템은 Google Gemini API를 활용하여 창의적 문제해결(Creative Problem Solving, CPS) 과정에서 학습자의 사고를 촉진하는 질문 기반 스캐폴딩을 제공합니다.

핵심 설계 원칙:

- 답변 제공이 아닌 사고 촉진
- 단계 강제 없는 자율적 진행
- 맥락 기반 적응적 질문 생성
- 메타인지 요소 기반 개입

2. Backend Architecture Overview

2.1 전체 시스템 구조

Backend는 계층화된 아키텍처(Layered Architecture)로 설계되었습니다:

```
┌──────────────────────────────────────────────────────────┐
│        API Layer (FastAPI)                               │
│  - REST Endpoints (/api/chat, /api/research)             │
│  - Request Validation (Pydantic)                         │
│  - CORS Middleware                                       │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│        Service Layer                                     │
│  - GeminiService (LLM Integration)                       │
│  - Prompt Engineering Logic                              │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│      Business Logic Layer                                │
│  - CPS Stage Detection                                   │
│  - Metacognitive Element Analysis                        │
│  - Response Depth Assessment                             │
│  - Turn Counting & Limit Management                      │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│        Data Layer (SQLAlchemy ORM)                       │
```

```
│  - Session Management (CRUD)          │
│  - Conversation Logging               │
│  - Metrics Aggregation                │
└──────────────────────────────────────┘
                │
                ▼
┌──────────────────────────────────────┐
│    Database (PostgreSQL/SQLite)       │
│  - Sessions, Conversations            │
│  - StageTransitions, SessionMetrics   │
└──────────────────────────────────────┘
```

2.2 계층별 책임 분리

API Layer:

- HTTP 요청/응답 처리
- 입력 데이터 검증 (Pydantic schemas)
- CORS 정책 관리
- GZip 압축 적용

Service Layer:

- Gemini API 통합
- System Prompt 구성
- LLM 응답 파싱 및 정규화

Business Logic Layer:

- CPS 단계 추론
- 메타인지 요소 분류
- 응답 깊이 평가
- 단계별 턴 제한 관리

Data Layer:

- ORM 기반 데이터베이스 추상화
- 세션 생명주기 관리
- 대화 히스토리 저장
- 연구용 데이터 수집

3. 핵심 로직 엔진 (Core Logic Engine)

3.1 GeminiService - LLM 통합 엔진

`/backend/app/services/gemini_service.py`

핵심 기능: 사용자 응답을 분석하여 CPS 단계와 메타인지 요소를 파악하고, 적절한 스캐폴딩 질문을 생성합니다.

주요 메서드:

```
def generate_scaffolding(
    user_message: str,
    conversation_history: List[Dict],
    current_stage: str
) -> Dict:
    """
    사용자 메시지에 대한 스캐폴딩 질문 생성

    Args:
        user_message: 학습자의 최신 응답
        conversation_history: 최근 5개 대화 맥락
        current_stage: 현재 CPS 단계

    Returns:
        {
            "current_stage": "도전_이해_자료탐색",
            "detected_metacog_needs": ["점검", "조절"],
            "response_depth": "shallow|medium|deep",
            "agent_message": "질문 텍스트",
            "should_transition": false,
            "reasoning": "추론 근거"
        }
    """
```

처리 흐름:

1. 컨텍스트 구성: 최근 5개 메시지만 선택 (토큰 효율성)
2. **Prompt** 결합: System Prompt + 대화 히스토리 + 사용자 응답
3. **Gemini API** 호출: gemini-2.0-flash-exp 모델 사용
4. **JSON** 파싱: LLM 응답에서 구조화된 데이터 추출
5. 타입 정규화: detected_metacog_needs를 배열로 변환
6. **Fallback** 처리: 에러 시 기본 질문 반환

컨텍스트 윈도우 관리:

```
MAX_CONTEXT_MESSAGES = 5  # 최근 5개 메시지만 유지

def _build_context(history: List[Dict], current_stage: str) -> str:
    recent_messages = history[-MAX_CONTEXT_MESSAGES:] if len(history) >
MAX_CONTEXT_MESSAGES else history
```

```
    context_lines = []
    for msg in recent_messages:
        role = "학생" if msg["role"] == "user" else "에이전트"
        context_lines.append(f"{role}: {msg['content']}")

    return "\n".join(context_lines)
```

설계 근거: 긴 대화에서도 토큰 사용량을 제한하면서 충분한 맥락을 유지합니다.

3.2 Prompt Engineering 전략

`/backend/app/services/gemini_service.py` (Lines 40-153)

System Prompt 구조:

1. 역할 정의:

```
당신은 대학생 예비교사의 창의적 사고를 촉진하는 사고 촉진자(thinking facilitator)입니다.
직접적인 답변이나 해결책을 제공하지 않고, 학습자가 스스로 생각할 수 있도록 질문을 통해
안내합니다.
```

2. **CPS** 단계 설명: 3단계 9개 세부 단계 정의
   - 도전 이해: 기회 구성, 자료 탐색, 문제 구조화
   - 아이디어 생성: 아이디어 발산, 아이디어 선택
   - 실행 준비: 해결책 고안, 수용 구축
3. 메타인지 요소 정의:
   - 점검 **(Monitoring)**: 과제 특성 평가, 수행 예측, 아이디어 판단
   - 조절 **(Control)**: 과제 참여 결정, 전략 선택/변경, 해결책 선택
   - 지식 **(Knowledge)**: 선행 경험 활성화, 메타인지 지식 업데이트
4. 행동 규칙:
   - 직접 답변 금지
   - 강제 단계 전환 금지
   - 응답 깊이 평가 필수
   - 1-2문장 이내 질문 생성
5. 출력 형식 지정:

```
{
  "current_stage": "도전_이해_자료탐색",
  "detected_metacog_needs": ["점검"],
  "response_depth": "shallow",
  "agent_message": "그 문제의 난이도는 어느 정도라고 판단되나요?",
  "should_transition": false,
  "reasoning": "학습자가 문제를 언급했으나 구체적인 분석이 부족하여 점검 활동 필요"
```

```
}
```

핵심 프롬프트 원칙:

- 단일 메타인지 요소 강제: 한 번에 하나의 메타인지 요소만 다룸
- 개방형 질문 원칙: Yes/No 질문 금지
- **CPS** 비선형 강조: 이전 단계 재방문 허용
- 자율성 보장: 학습자가 단계 전환 주도

3.3 응답 깊이 평가 (Response Depth Assessment)

평가 기준:

- **Shallow**: 짧은 응답(50자 이하), 구체성 부족, 단일 관점
- **Medium**: 적절한 길이(50-150자), 일부 구체적 내용, 2-3개 아이디어
- **Deep**: 긴 응답(150자 이상), 다양한 관점, 구체적 예시, 선행 경험 연결

활용 방법:

- Shallow → 동일 메타인지 요소로 추가 탐색
- Medium → 다음 메타인지 요소로 진행
- Deep (2회 이상) → 단계 전환 고려

3.4 CPS 단계 추론 로직

/backend/app/api/chat.py

자동 단계 감지:

```
transition_keywords = {
    "아이디어": "아이디어_생성",
    "아이디어 생성": "아이디어_생성",
    "다음 단계": None,  # 현재 단계의 다음 단계로
    "실행 준비": "실행_준비",
    "실행": "실행_준비",
}

transition_indicators = [
    "넘어가" in user_message_lower,
    "이동" in user_message_lower,
    "가자" in user_message_lower,
    "진행" in user_message_lower,
]
```

전환 우선순위:

1. 사용자 명시적 요청 (키워드 감지)

2. Gemini의 should_transition=true 권고
3. 최소 요구사항 충족 (예: 원인 3개 이상)

단계 전환 제약:

- 역방향 전환 불가 (도전 이해 ← 아이디어 생성)
- 순차적 진행 권장 (건너뛰기 방지)

4. 상태 관리 모듈 (State Management Module)

4.1 Database Schema

/backend/app/models/database.py

핵심 테이블 구조:

4.1.1 Sessions Table

```
class Session(Base):
    __tablename__ = "sessions"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String(255), nullable=True)  # 익명 허용
    assignment_text = Column(Text, nullable=False)  # 과제 텍스트
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    conversations = relationship("Conversation", back_populates="session")
    stage_transitions = relationship("StageTransition", back_populates="session")
    metrics = relationship("SessionMetric", back_populates="session", uselist=False)
```

역할: 세션 생명주기 관리 (Create → Active → Complete)

4.1.2 Conversations Table

```
class Conversation(Base):
    __tablename__ = "conversations"

    id = Column(Integer, primary_key=True, index=True)
    session_id = Column(Integer, ForeignKey("sessions.id"), nullable=False)
    role = Column(String(50), nullable=False)  # "user" or "agent"
    message = Column(Text, nullable=False)
    cps_stage = Column(String(50))  # "도전_이해_자료탐색" etc.
    metacog_elements = Column(JSONB)  # ["점검", "조절"]
    response_depth = Column(String(20))  # "shallow", "medium", "deep"
```

```
    created_at = Column(DateTime, default=datetime.utcnow)
```

역할: 모든 대화 기록과 CPS 주석 저장 (연구 데이터)

4.1.3 SessionMetrics Table

```
class SessionMetric(Base):
    __tablename__ = "session_metrics"

    id = Column(Integer, primary_key=True, index=True)
    session_id = Column(Integer, ForeignKey("sessions.id"), unique=True)

    # 단계별 턴 카운트
    challenge_understanding_turns = Column(Integer, default=0)  # 최대 6턴
    idea_generation_turns = Column(Integer, default=0)          # 최대 8턴
    action_preparation_turns = Column(Integer, default=0)       # 최대 6턴
    current_stage = Column(String(50))

    # 응답 깊이 분포
    shallow_responses = Column(Integer, default=0)
    medium_responses = Column(Integer, default=0)
    deep_responses = Column(Integer, default=0)

    # 메타인지 요소 빈도
    monitoring_count = Column(Integer, default=0)
    control_count = Column(Integer, default=0)
    knowledge_count = Column(Integer, default=0)

    last_updated = Column(DateTime, default=datetime.utcnow)
```

역할: 세션별 집계 지표 관리 (성능 최적화)

4.1.4 StageTransitions Table

```
class StageTransition(Base):
    __tablename__ = "stage_transitions"

    id = Column(Integer, primary_key=True, index=True)
    session_id = Column(Integer, ForeignKey("sessions.id"))
    from_stage = Column(String(50))
    to_stage = Column(String(50))
    transition_reason = Column(Text)  # 전환 사유
    created_at = Column(DateTime, default=datetime.utcnow)
```

역할: CPS 단계 전환 이력 추적 (학습 패턴 분석)

4.2 턴 카운팅 시스템 (Turn Counting System)

/backend/app/crud/session_metrics.py

단계별 턴 제한:

```
TURN_LIMITS = {
    "도전_이해": 6,
    "아이디어_생성": 8,
    "실행_준비": 6,
}
```

턴 업데이트 로직:

```
def update_turn_count(db: Session, session_id: int, cps_stage: str):
    metric = get_or_create_session_metric(db, session_id)

    # 단계별 컬럼 매핑
    stage_column_map = {
        "도전_이해": "challenge_understanding_turns",
        "아이디어_생성": "idea_generation_turns",
        "실행_준비": "action_preparation_turns",
    }

    column_name = stage_column_map.get(cps_stage.split("_")[0] + "_" + cps_stage.split("_")[1])

    # 카운트 증가
    current_value = getattr(metric, column_name)
    new_value = current_value + 1
    setattr(metric, column_name, new_value)

    metric.current_stage = cps_stage
    metric.last_updated = datetime.utcnow()
    db.commit()

    # 제한 도달 확인
    max_turns = TURN_LIMITS.get(cps_stage.split("_")[0] + "_" + cps_stage.split("_")[1], 999)
    limit_reached = new_value >= max_turns

    return new_value, max_turns, limit_reached
```

중요 설계 결정: 턴 제한 도달 시 자동 전환하지 않고, 학습자에게 알림만 제공하여 자율성을 보장합니다.

4.3 Connection Pool 설정

/backend/app/db/session.py

PostgreSQL 연결 풀:

```
from sqlalchemy import create_engine
from sqlalchemy.pool import QueuePool

engine = create_engine(
    settings.DATABASE_URL,
    poolclass=QueuePool,
    pool_size=5,            # 기본 연결 5개
    max_overflow=10,        # 추가 연결 최대 10개 (총 15개)
    pool_pre_ping=True,     # 연결 상태 확인
    pool_recycle=3600,      # 1시간마다 연결 재활용
    echo=settings.DEBUG,    # SQL 로깅 (개발 모드)
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

동시 사용자 지원 계산:

- 5개 기본 연결 + 10개 추가 = 최대 15개 연결
- 평균 응답 시간 0.5초 가정
- 초당 처리: 15 / 0.5 = 30 요청/초
- 예상 동시 사용자: 약 30명

4.4 Session Lifecycle Management

세션 생성:

```
POST /api/chat/session
{
    "assignment_text": "학생들의 수업 참여를 높이는 방법"
}
→ Returns: { "session_id": 123 }
```

대화 진행:

```
POST /api/chat/message
{
```

```
  "session_id": 123,
  "message": "학생들이 수업에 집중하지 못하는 이유는...",
  "conversation_history": [...],
  "current_stage": "도전_이해_자료탐색"
}
→ Returns: { "agent_message": "...", "scaffolding_data": {...} }
```

세션 종료: 별도 종료 API 없음 (자동 완료 감지)

5. 기술 스택 (Technology Stack)

5.1 Core Dependencies

/backend/requirements.txt

Web Framework:

- fastapi==0.109.0 - 비동기 웹 프레임워크
- uvicorn==0.27.0 - ASGI 서버
- pydantic==2.5.3 - 데이터 검증

LLM Integration:

- google-generativeai==0.3.2 - Gemini API 클라이언트

Database:

- sqlalchemy==2.0.25 - ORM
- psycopg2-binary==2.9.9 - PostgreSQL 드라이버
- alembic==1.13.1 - 마이그레이션 도구

Utilities:

- python-dotenv==1.0.0 - 환경 변수 관리
- python-multipart==0.0.6 - 파일 업로드 지원

5.2 선택 근거

FastAPI 선택 이유:

1. 비동기 지원 (높은 동시성)
2. 자동 API 문서 생성 (Swagger UI)
3. Pydantic 통합 (타입 안전성)
4. 빠른 개발 속도

Gemini API 선택 이유:

1. 긴 컨텍스트 윈도우 (1M 토큰)
2. JSON 모드 지원

3. 한국어 성능 우수
4. 무료 티어 제공

PostgreSQL 선택 이유:

1. JSONB 타입 지원 (메타인지 요소 저장)
2. 강력한 인덱싱
3. 연구 데이터 무결성 보장
4. 복잡한 쿼리 지원

5.3 Development vs Production

Development:

- SQLite (간단한 로컬 개발)
- Debug 로깅 활성화
- CORS 모든 출처 허용

Production:

- PostgreSQL (Railway 호스팅)
- 최소 로깅
- CORS 특정 도메인만 허용
- GZip 압축 활성화

6. 에이전트 작동 방식 (Agent Operation Method)

6.1 전체 처리 흐름

```
[사용자 메시지 입력]
      ↓
[1. Message Validation]
  - Pydantic schema 검증
  - session_id 존재 확인
      ↓
[2. Context Loading]
  - 최근 5개 대화 로드
  - 현재 CPS 단계 확인
  - 턴 카운트 조회
      ↓
[3. Stage Transition Detection]
  - 키워드 감지 ("아이디어", "다음 단계")
  - 명시적 전환 요청 처리
      ↓
[4. LLM Processing (GeminiService)]
  - System Prompt 구성
  - 대화 맥락 추가
```

```
  - Gemini API 호출
  - JSON 파싱
      ↓
[5. Response Analysis]
  - CPS 단계 추론
  - 메타인지 요소 분류
  - 응답 깊이 평가
  - 전환 여부 결정
      ↓
[6. Turn Count Update]
  - 단계별 카운트 증가
  - 제한 도달 확인
      ↓
[7. Database Persistence]
  - Conversation 레코드 저장 (user + agent)
  - SessionMetrics 업데이트
  - StageTransition 기록 (전환 시)
      ↓
[8. Response Construction]
  - agent_message (질문)
  - scaffolding_data (메타데이터)
  - 프론트엔드로 반환
```

6.2 의사결정 트리 (Decision Tree)

단계 전환 결정:

```
사용자 메시지 수신
  ↓
┌──────────────────────────────────────────┐
│ 명시적 전환 키워드 있음?                  │
│ ("아이디어", "다음 단계")                 │
└──────────┬──────────────────────────────┬──────
   YES                                    NO
   ↓                                      ↓
 즉시 전환                          ┌──────────────────┐
                                    │ 응답 깊이 평가   │
                                    └──────────┬───────┘
                                               ↓
                                    ┌──────────────────────┐
                                    │ Deep 응답 2회 이상?  │
                                    └──────────┬───────────┘
                                               YES
                                               ↓
```

```
┌──────────────────────────┐
│ 최소 요구사항 충족?  │
│ (원인 3개, 아이디어 5개 등)│
└──────────┬───────────────┘
           YES
            ↓
┌──────────────────────────┐
│ Gemini 전환 권고?    │
│ (should_transition=true)│
└──────────┬───────────────┘
           YES
            ↓
         단계 전환
            ↓
      StageTransition 기록
```

메타인지 요소 선택:

```
현재 CPS 단계 확인
  ↓
┌──────────────────────────────────────┐
│ 도전 이해 단계?              │
└──────┬──────────────────────────────┘
  YES              NO
   ↓               ↓
패턴: 지식 → 점검 → 조절  ┌──────────────────────┐
(반복)              │ 아이디어 생성?   │
                    └──────────┬───────────┘
                               YES
                                ↓
                          패턴: 점검 → 조절
                          (반복)
                                ↓
                    ┌──────────────────────┐
                    │ 실행 준비?       │
                    └──────────┬───────────┘
                               YES
                                ↓
                          패턴: 점검 → 조절 → 지식
                          (순차 진행)
```

6.3 Prompt Engineering 세부 전략

동적 **Prompt** 구성:

```
def _build_dynamic_prompt(
    user_message: str,
    conversation_history: List[Dict],
    current_stage: str,
    turn_count: int,
    max_turns: int
) -> str:
    # 1. Base System Prompt
    prompt = BASE_SYSTEM_PROMPT

    # 2. 단계별 안내 추가
    stage_guidance = STAGE_SPECIFIC_GUIDANCE[current_stage]
    prompt += f"\n\n현재 단계 안내:\n{stage_guidance}"

    # 3. 턴 제한 정보 추가
    if turn_count >= max_turns - 2:
        prompt += f"\n\n⚠️ 현재 단계에서 {turn_count}/{max_turns} 턴 진행됨. 다음 단계 전환 고려 필요."

    # 4. 대화 맥락 추가
    context = _build_context(conversation_history, current_stage)
    prompt += f"\n\n대화 맥락:\n{context}"

    # 5. 현재 사용자 메시지 추가
    prompt += f"\n\n학생의 최신 응답: \"{user_message}\""

    # 6. 출력 형식 지시
    prompt += "\n\nJSON 형식으로 응답하세요."

    return prompt
```

Few-Shot Examples 적용:

System Prompt에 각 CPS 단계별 모범 질문 예시 포함:

```
도전 이해 - 지식 활성화 예시:
- "이와 비슷한 문제를 경험한 적이 있나요?"
- "이 문제와 관련하여 알고 있는 배경 지식은 무엇인가요?"

도전 이해 - 점검 예시:
- "이 문제의 난이도는 어느 정도라고 생각하나요?"
- "문제의 핵심이 무엇이라고 판단되나요?"
```

```
도전 이해 - 조절 예시:
- "어떤 관점에서 이 문제를 바라보고 싶나요?"
- "문제를 해결하기 위해 어떤 정보가 더 필요할까요?"
```

6.4 에러 처리 및 Fallback 메커니즘

Gemini API 에러 처리:

```
try:
    response = model.generate_content(prompt)
    result = json.loads(clean_json_response(response.text))
except json.JSONDecodeError:
    # JSON 파싱 실패 시 기본 응답
    result = {
        "current_stage": current_stage,
        "detected_metacog_needs": ["점검"],
        "response_depth": "medium",
        "agent_message": "방금 말씀하신 내용을 조금 더 구체적으로 설명해주시겠어요?",
        "should_transition": False,
        "reasoning": "LLM 응답 파싱 실패, 기본 질문 제공"
    }
except Exception as e:
    # 기타 에러 시 안전한 fallback
    logger.error(f"Gemini API error: {e}")
    result = {
        "agent_message": "죄송합니다. 잠시 문제가 발생했습니다. 다시 시도해주시겠어요?",
        ...
    }
```

데이터베이스 트랜잭션 관리:

```
def save_conversation(db: Session, session_id: int, messages: List[Dict]):
    try:
        # 트랜잭션 시작
        for msg in messages:
            conversation = Conversation(
                session_id=session_id,
                role=msg["role"],
                message=msg["content"],
                cps_stage=msg.get("current_stage"),
                metacacog_elements=msg.get("metacog_elements"),
            )
```

```
        db.add(conversation)

    db.commit()
  except Exception as e:
    db.rollback()  # 에러 시 롤백
    logger.error(f"Database error: {e}")
    raise
```

7. 성능 및 확장성 (Performance & Scalability)

7.1 응답 시간 최적화

측정 기준:

- API 엔드포인트 응답 시간: 평균 2-3초 목표
- Gemini API 호출: 1-2초
- 데이터베이스 쿼리: 100ms 이하

최적화 전략:

1. 컨텍스트 윈도우 제한: 최근 5개 메시지만 사용
2. 연결 풀 재사용: 데이터베이스 연결 유지
3. 인덱스 활용: `session_id`, `created_at` 컬럼 인덱싱
4. **GZip** 압축: 응답 크기 30-50% 감소

7.2 동시 사용자 지원

현재 구성:

- Connection Pool: 5 + 10 (overflow) = 15개
- 예상 처리량: 30 요청/초
- 동시 사용자: 약 30명

확장 방안:

1. 수평 확장: 여러 인스턴스 배포 (Load Balancer)
2. 연결 풀 확대: `pool_size=10`, `max_overflow=20`
3. 캐싱 도입: Redis 기반 세션 캐시
4. 비동기 작업: Celery 기반 백그라운드 작업

7.3 토큰 사용량 관리

Gemini API 토큰 소비:

- System Prompt: 약 1,500 토큰
- 5개 대화 맥락: 약 500-1,000 토큰
- 사용자 메시지: 100-300 토큰
- 총 입력: 약 2,000-3,000 토큰/요청

- 출력: 약 200-400 토큰/요청

월간 예상 사용량 (30명 동시 사용자):

- 1세션당 평균 20회 대화
- 1회당 3,000 토큰 → 60,000 토큰/세션
- 30명 × 60,000 = 1,800,000 토큰/일
- 월간: 약 54M 토큰

비용 최적화:

- 무료 티어: 50 요청/분, 1,500 요청/일
- 유료 전환 고려 시점: 일 1,500 요청 초과

8. 보안 및 개인정보 (Security & Privacy)

8.1 데이터 보호

개인 식별 정보 최소화:

- `user_id` 필드는 선택 사항 (익명 사용 가능)
- 이메일, 이름 등 수집하지 않음
- IP 주소 로깅하지 않음

데이터베이스 보안:

- PostgreSQL 연결 암호화 (SSL)
- 환경 변수로 민감 정보 관리 (.env)
- Railway 자동 백업 활성화

8.2 API 보안

CORS 정책:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.allowed_origins_list,  # 프론트엔드 도메인만 허용
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Rate Limiting (향후 적용):

- IP 기반: 100 요청/분
- Session 기반: 50 요청/세션

8.3 연구 윤리

데이터 사용 동의:

- 세션 생성 시 연구 목적 안내
- 데이터 수집 동의 체크박스 (프론트엔드)

데이터 익명화:

- Export API는 `session_id`만 노출
- `user_id` 필드는 해시 처리 또는 제거

데이터 보관 기간:

- 연구 종료 후 6개월 보관
- 자동 삭제 스크립트 구현 예정

9. 향후 개선 사항 (Future Improvements)

9.1 기능 개선

1. 실시간 피드백: WebSocket 기반 스트리밍 응답
2. 다중 세션 관리: 학습자가 여러 과제 동시 진행
3. 성과 대시보드: 메타인지 활동 시각화
4. 교수자 모드: 학습자 진행 상황 모니터링

9.2 성능 개선

1. 캐싱 계층: Redis 기반 세션 캐시
2. 백그라운드 작업: Celery로 분석 작업 비동기 처리
3. **CDN** 통합: 정적 리소스 캐싱
4. 로드 밸런싱: 여러 백엔드 인스턴스 분산

9.3 연구 기능 확장

1. **A/B** 테스트 프레임워크: 스캐폴딩 전략 실험
2. 학습 분석: 메타인지 패턴 자동 탐지
3. 비교 그룹 관리: 대조군 vs 실험군
4. 데이터 **Export**: SPSS, R 호환 형식

10. 기술 결정 근거 (Technical Decision Rationale)

10.1 왜 FastAPI인가?

대안:

- Flask: 경량, 단순
- Django: Full-stack, ORM 내장

선택 이유:

- 비동기 지원으로 동시성 처리 우수

- Pydantic 통합으로 타입 안전성 확보
- 자동 API 문서 (Swagger UI) 생성
- 빠른 개발 속도와 생산성

10.2 왜 Gemini API인가?

대안:

- OpenAI GPT-4: 성능 우수, 비용 높음
- Anthropic Claude: 긴 컨텍스트, 한국어 제한적
- Llama (Self-hosted): 비용 없음, 인프라 복잡

선택 이유:

- 1M 토큰 컨텍스트 (긴 대화 지원)
- JSON 모드 네이티브 지원
- 한국어 성능 우수
- 무료 티어 충분 (개발/연구 단계)

10.3 왜 PostgreSQL인가?

대안:

- MySQL: 널리 사용됨
- MongoDB: NoSQL, 유연한 스키마
- SQLite: 파일 기반, 간단

선택 이유:

- JSONB 타입 (메타인지 요소 저장)
- 복잡한 쿼리 최적화
- Railway 네이티브 지원
- 연구 데이터 무결성 보장

10.4 왜 계층화 아키텍처인가?

대안:

- Monolithic: 단순 구조
- Microservices: 완전 분리
- Serverless: FaaS 기반

선T택 이유:

- 관심사 분리 (Separation of Concerns)
- 테스트 용이성
- 향후 마이크로서비스 전환 가능
- 중간 규모 프로젝트에 적합

11. 결론 및 요약 (Conclusion & Summary)

11.1 핵심 성과

본 Backend 시스템은 다음과 같은 핵심 기능을 구현했습니다:

1. **Gemini API** 통합: Prompt Engineering 기반 맥락 인식 질문 생성
2. **CPS** 단계 자동 추론: 학습자 응답 기반 단계 전환 지원
3. 메타인지 요소 분류: 점검, 조절, 지식 요소 자동 감지
4. 턴 카운팅 시스템: 단계별 진행 추적 및 제한 관리
5. 연구 데이터 수집: PostgreSQL 기반 구조화된 데이터 저장
6. 확장 가능한 아키텍처: 계층화 설계로 유지보수성 확보

11.2 시스템 특징

- 자율성 보장: 강제 전환 없는 학습자 주도 진행
- 맥락 인식: 최근 5개 대화 기반 적응적 질문 생성
- 효율적 리소스 관리: 연결 풀, 컨텍스트 윈도우 제한
- 연구 친화적: 모든 상호작용 로깅 및 Export 기능

11.3 기술 스택 요약

계층	기술	역할
Web Framework	FastAPI 0.109.0	비동기 API 서버
LLM	Google Gemini 2.0	질문 생성 엔진
Database	PostgreSQL (Railway)	데이터 저장
ORM	SQLAlchemy 2.0.25	데이터 추상화
Server	Uvicorn 0.27.0	ASGI 실행 환경

11.4 성능 지표

- 응답 시간: 평균 2-3초
- 동시 사용자: 약 30명 지원

- 컨텍스트 효율: 5개 메시지 제한으로 토큰 절약
- 데이터베이스 효율: 연결 풀 15개, 30 요청/초 처리

11.5 향후 발전 방향

1. 실시간 스트리밍: WebSocket 기반 점진적 응답
2. 고급 분석: 학습 패턴 자동 탐지 및 시각화
3. 다중 **LLM** 지원: Gemini 외 GPT-4, Claude 통합
4. 교수자 대시보드: 학습자 진행 상황 실시간 모니터링
5. **A/B** 테스트: 스캐폴딩 전략 실험 프레임워크

부록 **A:** 환경 변수 설정

```
# .env 파일 예시

# Gemini API
GEMINI_API_KEY=your_api_key_here

# Database (Development)
DATABASE_URL=sqlite:///./univ_consult.db

# Database (Production)
DATABASE_URL=postgresql://user:password@host:5432/dbname

# Application
DEBUG=false
LOG_LEVEL=info
ALLOWED_ORIGINS=[https://your-frontend-domain.com](https://your-frontend-domain.com)

# Uvicorn
UVICORN_WORKERS=2
```

부록 **B: API** 엔드포인트 목록

Chat API (/api/chat):

- POST /session - 새 세션 생성
- POST /message - 메시지 전송 및 스캐폴딩 받기

Research API (/api/research):

- GET /sessions - 모든 세션 조회
- GET /sessions/{session_id}/conversations - 대화 히스토리
- GET /sessions/{session_id}/metrics - 세션 지표
- GET /export/conversations/csv - 대화 CSV Export
- GET /export/metrics/csv - 지표 CSV Export